



API SOP

VERSION 1.0

COPYRIGHT NOTICE

Copyright © Yalamanchili International Pte Ltd.

All rights reserved. These materials are confidential and proprietary to Yalamanchili International Pte Ltd. and no part of these materials should be reproduced, published in any form by any means, electronic or mechanical including photocopy or any information storage or retrieval system or should the materials be in compliance to third parties without the express written authorization of Yalamanchili International Pte Ltd.

NON-DISCLOSURE AGREEMENT

The reader of this document hereby undertakes to keep all information, documents, and knowledge of whatever kind obtained by them or by any of our advisers or agents (whether orally, in writing or in any way recorded or by observation (“the Information”), Yalamanchili International Pte Ltd. in connection with the proposals referred to in clause three below (hereafter referred to as “YALAMANCHILI”) secret and confidential and we further undertake, agree and acknowledge the following:

The reader will not (save with the prior written consent of YALAMANCHILI or as required by law or the relevant regulatory body) communicate any of the Information to any other person or body, (“except associated companies”), in connection with the proposals referred to in the clause below.

The reader will not make or permit or procure to be made or solicit or assist any other person to make any announcement or disclosure of our discussions with YALAMANCHILI, whether or not any discussions commence, proceed or are terminated, without YALAMANCHILI’s prior written consent except as required by law or competent regulatory body. If agreements to any cooperation with YALAMANCHILI, at or after the date of such cooperation, no announcement of the cooperation will be made except by mutual agreement between YALAMANCHILI and the reader or as required by law or competent regulatory body.

The reader will make no use of the Information for any purpose other than giving consideration to the possibility of putting forward proposals concerning a potential cooperation with YALAMANCHILI or for the purpose of such proposals, and, in particular, but without prejudice to the generality of the foregoing, the reader will not use the Information to obtain any commercial benefit or for any purpose detrimental to YALAMANCHILI.

Table Of Content

1. INTRODUCTION.....	5
1.1 Key Concepts of APIs.....	5
1.2 Types of APIs.....	6
1.3 Usage & Use case of APIs	6
2. SCOPE	9
3. OBJECTIVE.....	10
4. TOOLS REQUIRED	11
5. TEST CASES FOR INBOUND (SOAP) API	12
5.1 SQL Injections.	12
SQL INJECTION SENSITIVE FEILD	14
5.1.3 COMMAND INJECTION	16
5.1.4 XML INJECTION.....	18
5.2 SOAP Action Spoofing.....	20
5.3 SOAP Parameter DOS Attack.....	22
5.4 WSDL disclosure	24
5.5 Payload Encryption	25
6. REST API TEST CASES	26
6.1 Test for HTTP methods.....	26
6.2 Clear text Transmission of Sensitive Data	28
6.3 Performance / No Rate Limit Test.....	29
7. LIST OF COMMON TEST CASE FOR BOTH INBOUND (SOAP) API & REST API.....	31

Document Version History

S. No.	Section Changed	Amended By	Amended Date	Description of Change	Resultant Version No.	Origin of Change
1	Security Response Workflow	Yalamanchili	21/05/2025	Revised and clarified the workflow from attacker detection to IP blocking.	1.0	Initial creation of workflow

Revision History							
NO	Date of Release/ Revision	Prepared/ Revised By	Reviewed by Name	Date	Approved by Name	Date	Reason for Revision
1.0	29-May-25	Chandu Prakash	CISO(Chief Information Security Officer) Nelby Sir	21 st - May-25	COO & CEO	21-May-25	NA

1. Introduction

An **Application Programming Interface (API)** is a set of well-defined rules and protocols that enables different software systems to communicate with each other. Rather than exposing the internal workings of an application, an API provides a structured way for one system to request and receive information or services from another.

Think of an API as a **bridge** between different applications. It allows systems—often built with different technologies—to work together efficiently by following a shared language and format for requests and responses. This approach simplifies integration, improves compatibility across platforms, and supports scalable and modular application design.

To put it into a relatable context, an API functions much like a **waiter in a restaurant**. When you (the client) place an order, the waiter (API) takes that request to the kitchen (server), ensures it's prepared correctly, and then delivers the result (response) back to you. At no point do you need to know how the kitchen operates—you just receive the service you requested.

By standardizing these interactions, APIs play a crucial role in enabling everything from mobile apps and web services to large-scale enterprise integrations, making modern digital ecosystems more connected and efficient.

1.1 Key Concepts of APIs

Endpoints: Specific paths or URLs where an API can be accessed (e.g., GET /api/users).

- **Methods (Verbs):** Define what action to perform:
 - GET - Retrieve data
 - POST - Create new data
 - PUT/PATCH - Update existing data
 - DELETE - Remove data
- **Requests and Responses:** APIs accept requests (usually in JSON or XML) and return responses, typically containing data or status codes (e.g., 200 OK, 404 Not Found).
- **Authentication:** APIs often require tokens, API keys, or OAuth credentials to control access and security.

1.2 Types of APIs

➤ **RESTful APIs**

Use HTTP methods and are stateless. Most common type today due to simplicity and scalability.

➤ **SOAP APIs**

Use XML and have stricter standards. Often used in enterprise environments.

1.3 Usage & Use case of APIs

➤ **REST API (Representational State Transfer API)**

A **REST API** is a standardized architectural style used to design networked applications. It operates over HTTP/HTTPS and uses HTTP methods such as GET, POST, PUT, DELETE to interact with resources (typically in JSON format). REST APIs are stateless, scalable, and commonly used in web and mobile applications.

Use Case	Description
Web/Mobile Applications	A mobile app retrieves or submits user data (e.g., login, profile updates) using RESTful services.
CRUD Operations	Perform Create, Read, Update, and Delete operations on data entities in a database.
Third-party Integrations	REST APIs expose services to external clients (e.g., payment gateways, weather APIs).
Headless CMS / eCommerce	Frontends fetch content or product data from RESTful backends without tight coupling.

➤ **Inbound (SOAP) API**

An **Inbound API** refers to any API that allows **external systems or applications to initiate communication and send data into your system or application**. It is a direction-specific term indicating that your system is receiving (inbound) requests. In many cases, Inbound APIs are implemented using REST, SOAP, or GraphQL.

Use Case	Description
Webhook Listener	An inbound API receives real-time updates (e.g., payment status, form submissions) from external systems like Stripe, PayPal, or CRM tools.
Third-party Data Ingestion	Vendors or partners push data to your system (e.g., IoT sensors sending readings, external HR tools updating employee records).
API Gateways	External clients send requests to your backend services via an Inbound API exposed through a secure gateway.
ERP/CRM Integrations	External applications connect to your ERP or CRM using Inbound APIs to update records (e.g., customer data, inventory levels).
Customer Portals	APIs accept requests from external users logging in, retrieving invoices, or submitting support tickets.

➤ **Key Differences**

Feature	SOAP API	REST API
Message Format	XML only	JSON (commonly), XML, HTML
Transport Protocol	HTTP, SMTP, TCP, etc.	HTTP/HTTPS only
Description Language	Requires WSDL	No WSDL; uses endpoints and HTTP methods
Message Size	Heavy (XML, envelope)	Lightweight (typically JSON)
Security	WS-Security (built-in)	HTTPS, OAuth, JWT, etc.
Error Handling	SOAP Fault messages	HTTP status codes
Use Case	Enterprise, financial systems	Web, mobile, public APIs

➤ **Summary**

- SOAP is best suited for **enterprise-level**, **secure**, and **structured environments** where formal contracts (WSDL) and **reliable messaging** are required.
- REST is ideal for **web-based**, **stateless** communication and **public-facing APIs** due to its simplicity, speed, and **ease of integration**.

2. Scope

This scope defines the security testing boundaries for both **REST** and **Inbound (SOAP)** APIs, focusing on identifying risks that may affect data confidentiality, integrity, and availability in modern integrated systems.

➤ REST API Security Testing

REST APIs are commonly used for communication between web/mobile clients and backend services. The security scope includes:

- Validating **authentication** and **authorization** (e.g., OAuth 2.0, JWT).
- Testing for **injection vulnerabilities** (e.g., SQL, header injection).
- Enforcing **rate limiting**, **throttling**, and **quota controls**.
- Ensuring secure **error handling** and no sensitive data exposure.
- Verifying **HTTPS enforcement** and secure TLS configurations.
- Assessing **input validation** and **output encoding**.
- Mapping findings to **OWASP API Security Top 10** risks.

➤ Inbound (SOAP) API Security Testing:

SOAP APIs typically receive data from external systems and operate in high-trust, enterprise environments. The testing scope includes:

- Assessing **WS-Security** features (encryption, signing, tokens).
- Testing for **XML-based attacks** (e.g., XXE, XML injection).
- Verifying **schema validation** and message format enforcement.
- Preventing **replay attacks** via timestamp and nonce validation.
- Ensuring **authentication** and **operation-level access control**.
- Checking for **secure SOAP fault** messages.
- Enforcing **TLS encryption** and secure transmission protocols.

➤ Common Objectives

- Detect and mitigate vulnerabilities across API endpoints.
- Validate compliance with internal policies and industry standards (e.g., PCI-DSS, ISO/IEC 27001, OWASP API Security Top 10, SOC 2,).
- Ensure proper **logging**, **monitoring**, and **auditability** of API interactions

3. Objective

The objective of this document is to provide an in-depth understanding of **REST APIs** and **Inbound (SOAP) APIs** from an application security standpoint. The focus is on identifying potential security risks, implementing best practices for securing APIs, and ensuring the integrity, confidentiality, and availability of data being transmitted. Specifically, this document aims to:

- **Identify Security Risks:** Analyze and highlight the security vulnerabilities inherent in both **REST APIs** and **Inbound (SOAP) APIs**, such as improper authentication, authorization flaws, data leakage, and injection attacks.
- **Implement Secure API Design Practices:** Provide technical guidance on how to securely design and configure **REST APIs** and **Inbound (SOAP) APIs**, including the use of strong authentication mechanisms (e.g., OAuth 2.0, JWT), secure communication protocols (e.g., HTTPS), and data encryption techniques.
- **Mitigate Common API Vulnerabilities:** Offer strategies to address common API-related security threats, including:
 - **API Rate Limiting and Throttling** to prevent DoS attacks
 - **Input Validation** to mitigate injection attacks
 - **Access Control** to ensure proper user privileges
 - **Logging and Monitoring** to detect and respond to suspicious activity
- **Enforce Secure Communication:** Emphasize the importance of using **TLS/SSL encryption** for data in transit and secure authentication tokens to protect the data integrity and prevent man-in-the-middle (MITM) attacks.
- **Conduct Vulnerability Assessment:** Recommend tools and methods for continuous vulnerability scanning, penetration testing, and code review of **REST** and **Inbound APIs** to proactively identify and mitigate security flaws.
- **Ensure Compliance:** Discuss the importance of ensuring API security measures align with industry standards and regulatory requirements (e.g., ISO/IEC 27001, OWASP API Security Top 10, PCI-DSS, SOC 2).

By focusing on these areas, this document aims to equip security analysts with the knowledge and tools needed to assess and strengthen the security posture of **REST APIs** and **Inbound (SOAP) APIs**, ensuring the secure exchange of sensitive data and mitigating potential risks to organizational systems and user privacy.

4. Tools Required

- **For REST API Testing:** Tools like **Postman** is commonly used for **functional** and **automated** testing of **REST APIs**.
- **For Inbound (SOAP) API Testing:** **SoapUI** is the most widely used tool for testing **SOAP APIs**, offering extensive features for both **functional** and **security testing**. **Postman** can also be used for **SOAP API testing**, though it is primarily designed for **REST APIs**.
- **For Security Testing:** Tools like **Burp Suite** help assess both **REST** and **SOAP API security**, detecting vulnerabilities such as authentication flaws, cross-site scripting (XSS), and SQL injection.

This table can be included in an **SOP** document for API testing to clarify the appropriate tools to use for different types of **API testing**.

API Type	Tool	Purpose	Key Features	Use Case
REST API	Postman	Used for testing REST APIs , providing an interface to send requests and validate responses.	Request creation (GET, POST, PUT, DELETE), environment management, automated scripts	Functional testing of API endpoints, validating responses, and ensuring business logic.
Inbound (SOAP)	SoapUI	Comprehensive testing tool for both SOAP and REST APIs , commonly used for inbound (SOAP) APIs.	Data-driven testing, assertions for response validation, supports both SOAP & REST	Functional testing for SOAP APIs , ensuring proper request handling and correct response structure.
Security Testing	Burp Suite	Used for security testing of web applications and APIs.	Intercepting proxy, vulnerability scanning, fuzzing, session handling	Identifying security flaws in REST and SOAP APIs , such as authentication issues and data breaches.

5. Test Cases for Inbound (SOAP) API.

5.1 SQL Injections.

5.1.1 SQL INJECTION ON USERNAME AND PASSWORD FIELD

Test Case Name: SQL Injection on the username and password fields

Severity: **High**

DESCRIPTION:

SQL injection is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. It allows an attacker to view data that they are not normally able to retrieve. This might include data belonging to other users, or any other data that the application itself is able to access. In many cases, an attacker can modify or delete this data, causing persistent changes to the application's content or behaviour. As part of this test case, both the username and password fields are injected with values which will alter the query to be executed in the database. The user will be able to view all the users retrieved from the database.

Test Procedure:

1. Open the SOAP UI Tool.
2. Create a new SOAP project using file tab.
3. Now add WSDL URL and provide project name.
4. OPEN the folder and select one soap request.
5. Now open the request and inject SQL Query on username and password field.
6. For every request we need to change transaction reference number and proxy number.
7. Now verify the response code.

Image: It is observed that we inject SQL Query on username and password fields.

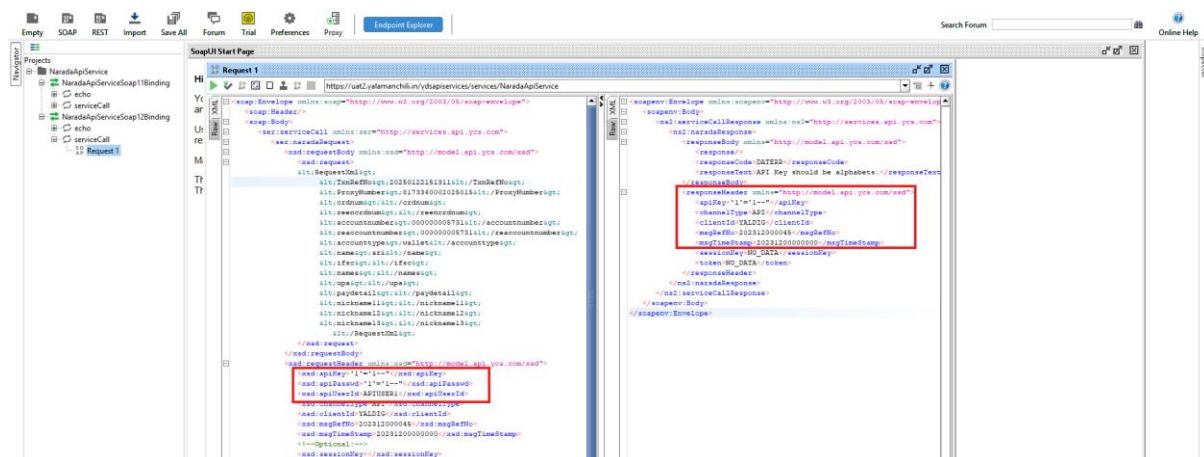
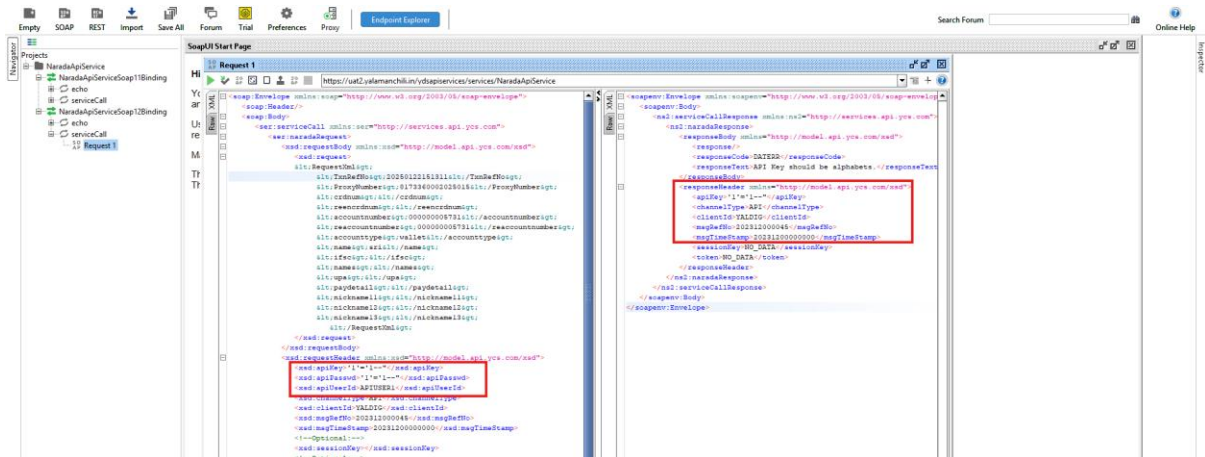


Image 2: Here it is observed that in response it shows DATERR (Data Error).



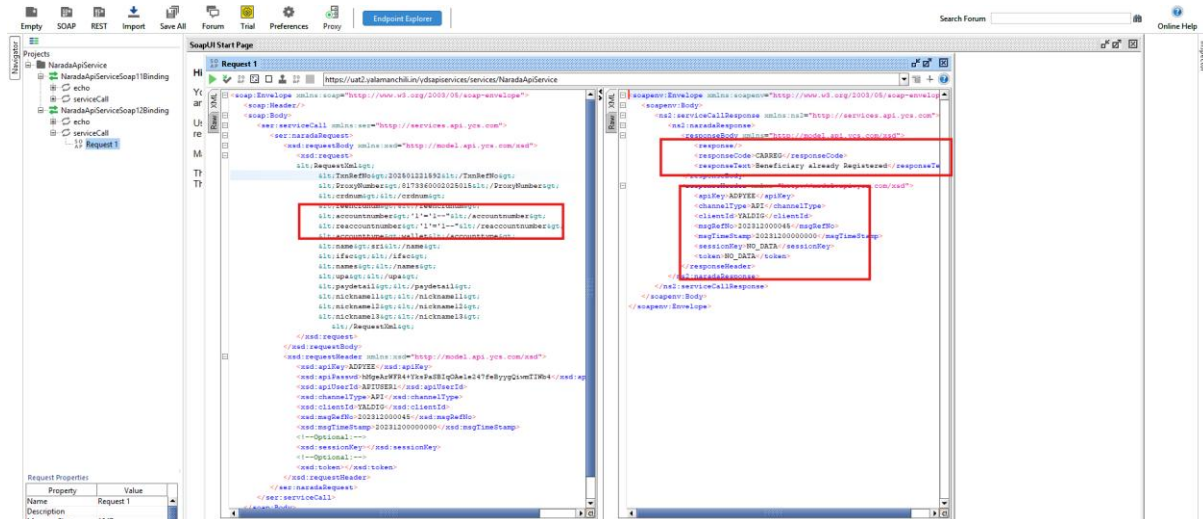
Expected Result:

If the response code is (transaction success) "00" , Then the cases is open otherwise the case is closed.

Mitigation:

Proper sanitization of data should be performed both at the UI and server level. use prepared statements, parameterized queries, or stored procedures instead whenever possible.

Image 2: It is observed that after passing the Query, the request responded properly. it shows valid CARREG (Beneficiary already registered) error code.



Expected Result:

The expected response is "00," indicating a successful transaction, along with an error such as "DATERR" (Data Error). In this case, we have determined that the request is already registered, so the case remains open for further review.

Mitigation:

The sensitive data must be properly validated before storing to the server.

5.1.3 COMMAND INJECTION

Test Case Name: COMMAND INJECTION

Severity: **High**

DESCRIPTION:

Command injection is an attack which is usually performed by passing a command with the data to get the different information like directory structures, sitemap or even sensible information related to the API or web application. This allows the attacker to trick the server into executing those commands on the underlying operating system.

Test Procedure:

1. Open the SOAP UI Tool.
2. Create a new SOAP project using file tab.
3. Now add WSDL URL and provide project name.
4. OPEN the folder and select one soap request.
5. Now open the request and add command injection payload.
6. For e.g.

```
<xsd:Test>
ws-hello-world.php
ws-lookup-dns-record.php
ws-user-account.php
</result>
</answer>
</xsd:Test>
```

7. For every request we need to change transaction reference number and proxy number.
8. Now verify the response code.

Image: Here we add command injection Payload

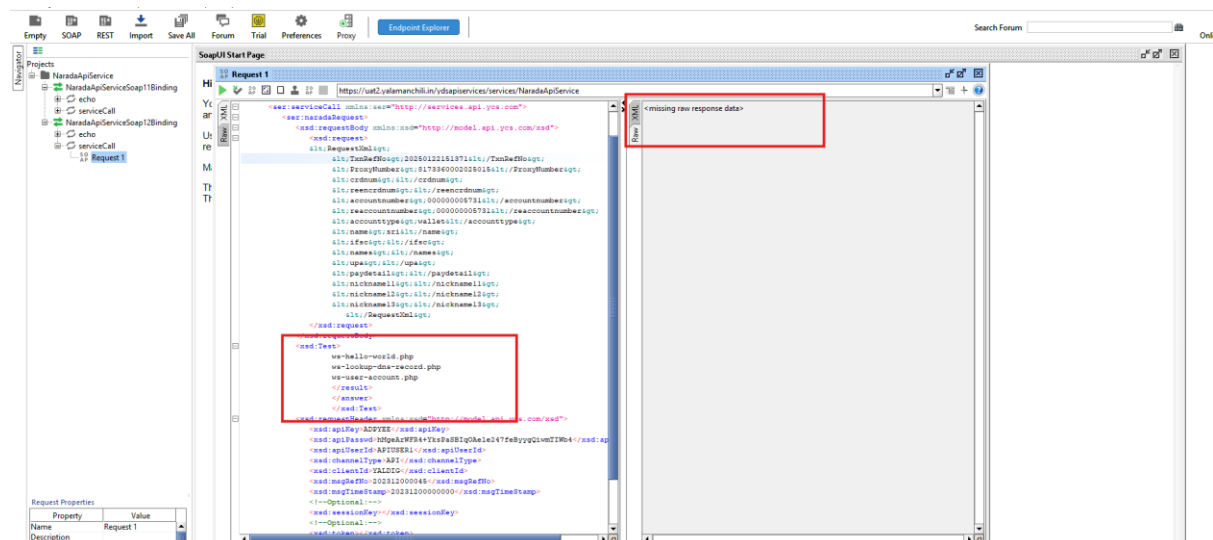
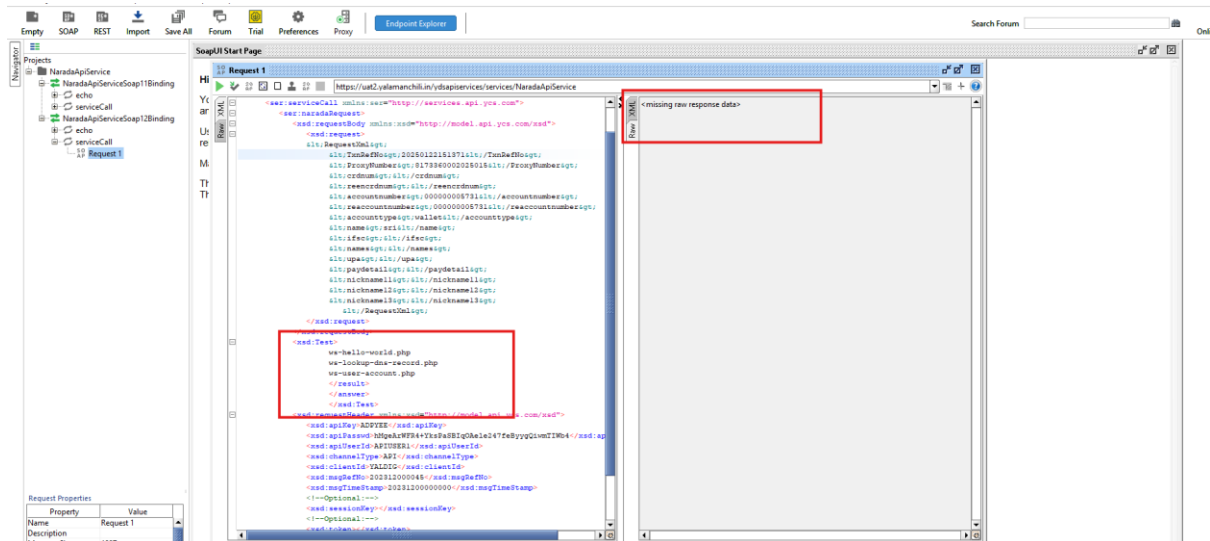


Image 2: After passing the payload we get an error in response tab.



Expected Result:

The Response should be “00” (transaction success). Or else if we can see any sensitive data related to API in the response then the case is open. Here in this case, we observed that we are unable to find the response, so the case is closed.

Mitigation:

Apply XML schema validation (XSD) to ensure the structure and type of the input are strictly enforced. Always **validate** and **sanitize** user inputs, especially file names, paths, or parameters passed to system functions. Keep security tight with authentication, logging, and regular checks

5.1.4 XML INJECTION

Test Case Name: XML INJECTION

Severity: High

DESCRIPTION:

In XML Injection type of attack, a morphed request of SOAP message can make changes in the database and cause serious damage to database. It is a security vulnerability that occurs when untrusted data is inserted into an XML structure without proper validation or sanitization, allowing an attacker to modify the intended logic or structure of the XML document.

Test Procedure:

1. Open the SOAP UI Tool.
2. Create a new SOAP project using file tab.
3. Now add WSDL URL and provide project name.
4. OPEN the folder and select one soap request.
5. Now open the request and add XML injection payload.
6. For e.g.

<urn:createUser sospenv:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

```
<username xsi:type="xsd:string">ganga</username>
<password xsi:type="xsd:string">name</password>
<signature xsi:type="xsd:string">alpha</signature>
</urn:createUser>
```

7. For every request we need to change transaction reference number and proxy number.
8. Now verify the response code.

Image: Here we inject XML Payload in request.

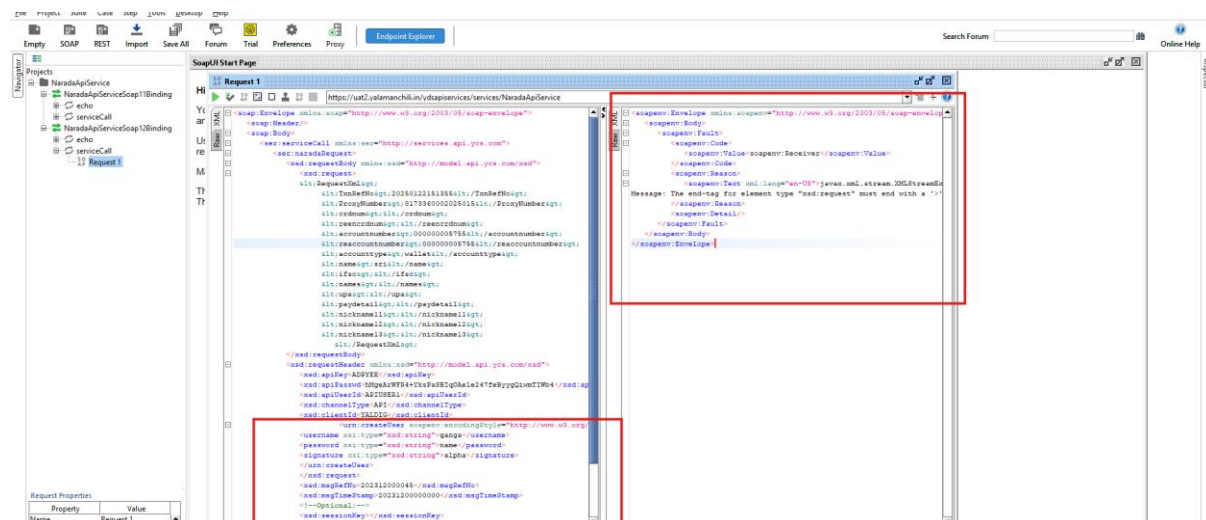
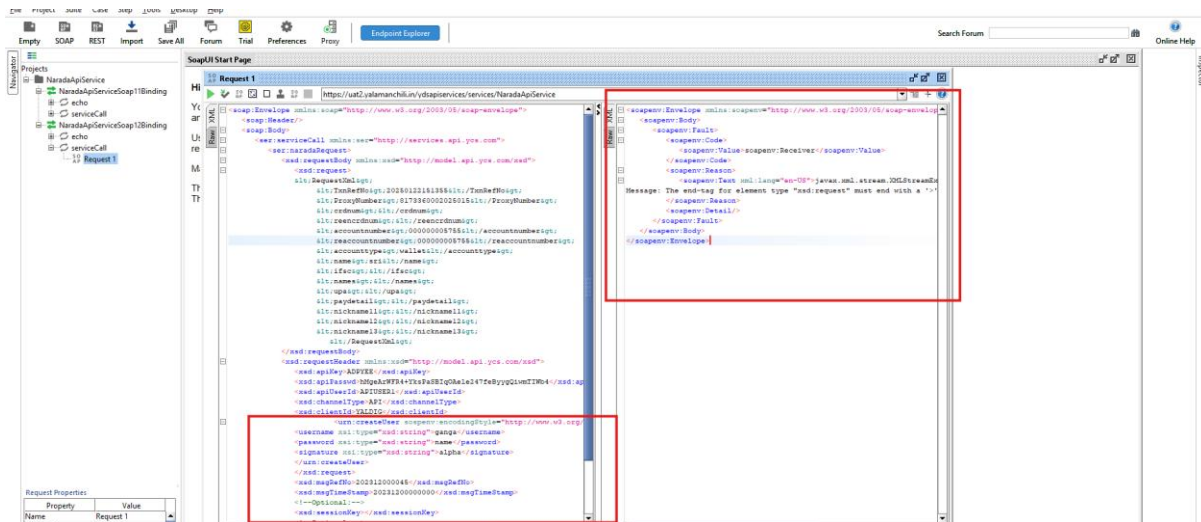


Image 2: It is observed that the response throws an error.



Expected Result:

If the response confirms that the user has been successfully created, the case remains open. Otherwise, if no user is found in the response, the case is considered closed.

Mitigation:

It's important to set clear limits on tag strings like `createUser`, defining both their allowed length and the number of times they can appear in the WSDL. Additionally, user input should be carefully sanitized based on the data format and parsing method used. Using **Document Type Definition (DTD)** for validation adds another layer of security, helping detect and block injection attempts while ensuring safe and reliable processing.

5.2 SOAP Action Spoofing

Test Case Name: Soap Action Spoofing

Severity: High

Description:

SOAP Action spoofing occurs when an attacker alters the SOAP Action header or body to invoke unauthorized operations. If the server fails to validate consistency between the SOAP header and body, it may execute unintended actions. This can be exploited in Man-in-the-Middle scenarios to bypass access controls or invoke privileged functions like createUser. Strict validation of SOAP headers and message content is essential to prevent spoofing and ensure secure API communication.

Test Procedure:

1. Open the SOAP UI Tool.
2. Create a new SOAP project using file tab.
3. Now add WSDL URL and provide project name.
4. OPEN the folder and select one soap request.
5. For every request we need to change transaction reference number and proxy number.
6. We can execute this only if there is a user creation functionality at API.
7. We need to manipulate request from user creation to user deletion.
8. Verify the Response if the user is deleted or not.

Image1: Here it is observed that the API has user creation functionality

```
POST /mutillidae/webservices/soap/ws-user-account.php HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:50.0) Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://192.168.66.129/mutillidae/webservices/soap/ws-user-account.php
Cookie: showhims=1; PHPSESSID=5ferut4eak7dp47nd4h6c0ghr5; JSESSIONID=AC970DC42FC77156F52646E60B17C15E
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
SOAPAction: urn:ws-user-account#createUser
Content-Type: text/xml; charset=UTF-8
Host: 192.168.66.129
Content-Length: 607

<soapenv:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001
xmlns:urn="urn:ws-user-account">
  <soapenv:Header/>
  <soapenv:Body>
    <urn:createUser soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <username xsi:type="xsd:string">gero et</username>
      <password xsi:type="xsd:string">sonoras imperio</password>
      <signature xsi:type="xsd:string">quae divum incedo</signature>
    </urn:createUser>
  </soapenv:Body>
</soapenv:Envelope>
```

Image 2: Here we alter the request from user creation to user deletion

```
POST /mutillidae/webservices/soap/ws-user-account.php HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:50.0) Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://192.168.66.129/mutillidae/webservices/soap/ws-user-account.php
Cookie: showhints=1; PHPSESSID=5ferut4eak7dp47nd4h6c0ghr5;
SESSIONID=A2970DC42FC77158F52846E60817C19E
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
SOAPAction: urn:ws-user-account#deleteUser
Content-Type: text/xml;charset=UTF-8
Host: 192.168.66.129
Content-Length: 607

<soapenv:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:urn="urn:ws-user-account">
  <soapenv:Header/>
  <soapenv:Body>
    <urn:createUser soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
      <username xsi:type="xsd:string">gero et</username>
      <password xsi:type="xsd:string">sonoras imperio</password>
      <signature xsi:type="xsd:string">quae divum incedo</signature>
    </urn:createUser>
  </soapenv:Body>
</soapenv:Envelope>
```

Image 3: It is observed that user is deleted. It shows 200 O.K response.

```
HTTP/1.1 200 OK
Date: Tue, 20 Dec 2016 09:03:30 GMT
Server: Apache/2.2.14 (Ubuntu) mod_mono/2.4.3 PHP/5.3.2-lubuntu4.30 with Suhosin-Patch
proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL/0.9.8k
Phusion Passenger/4.0.38 mod_perl/2.0.4 Perl/v5.10.1
X-Powered-By: PHP/5.3.2-lubuntu4.30
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
X-SOAP-Server: NuSOAP/0.9.5 (1.123)
Vary: Accept-Encoding
Content-Length: 560
Connection: close
Content-Type: text/xml; charset=ISO-8859-1

<?xml version="1.0" encoding="ISO-8859-1"?><SOAP-ENV:Envelope
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"><SOAP-ENV:Body><ns1:deleteUserR
esponse xmlns:ns1="urn:ws-user-account"><return xsi:type="xsd:xml"><accounts
message="Deleted account gero et"
/></return></ns1:deleteUserResponse></SOAP-ENV:Body></SOAP-ENV:Envelope>
```

Expected Result:

If the user is successfully deleted, the case remains open. However, if the deletion is unsuccessful, the case is considered closed.

Mitigation:

To prevent SOAP Action Spoofing, use unique and complex action identifiers instead of common names like **createUser** or **delete User**. If needed, set the **SOAP Action** header as empty and define the message intent through the URI. Always validate that the SOAP Action in the HTTP header matches the operation in the SOAP body. Strengthen security with authentication, authorization, and TLS/SSL encryption to prevent unauthorized access and interception.

5.3 SOAP Parameter DOS Attack

Test Case Name: SOAP Parameter DOS Attack

Severity: High

Description:

A SOAP Parameter Denial of Service (DoS) Attack occurs when an attacker overwhelms a SOAP API with excessively large or malformed requests, straining system resources and potentially causing service disruptions or crashes. This tactic exploits vulnerabilities in request handling, consuming memory and CPU power until the server becomes unresponsive. To prevent such attacks, it's crucial to limit SOAP message sizes, validate input properly, enforce timeouts, and continuously monitor server performance. Strengthening these defences ensures the API remains stable, responsive, and secure.

Test Procedure:

1. Open the SOAP UI Tool.
2. Create a new SOAP project using file tab.
3. Now add WSDL URL and provide project name.
4. OPEN the folder and select one soap request.
5. Now open the request and add XML injection payload.
6. For every request we need to change transaction reference number and proxy number.
7. We can execute this only if there is a user creation functionality at API.
8. We need to manipulate request from user creation to user deletion.
9. Verify the Response if the user is deleted or not.

Image: Here we add more data in username or password field.

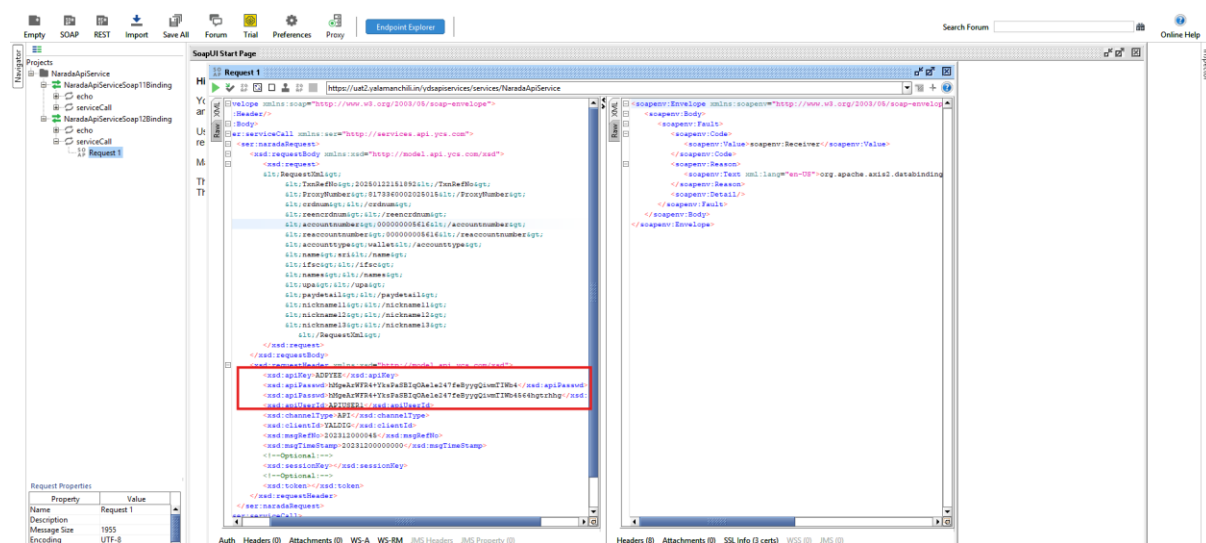
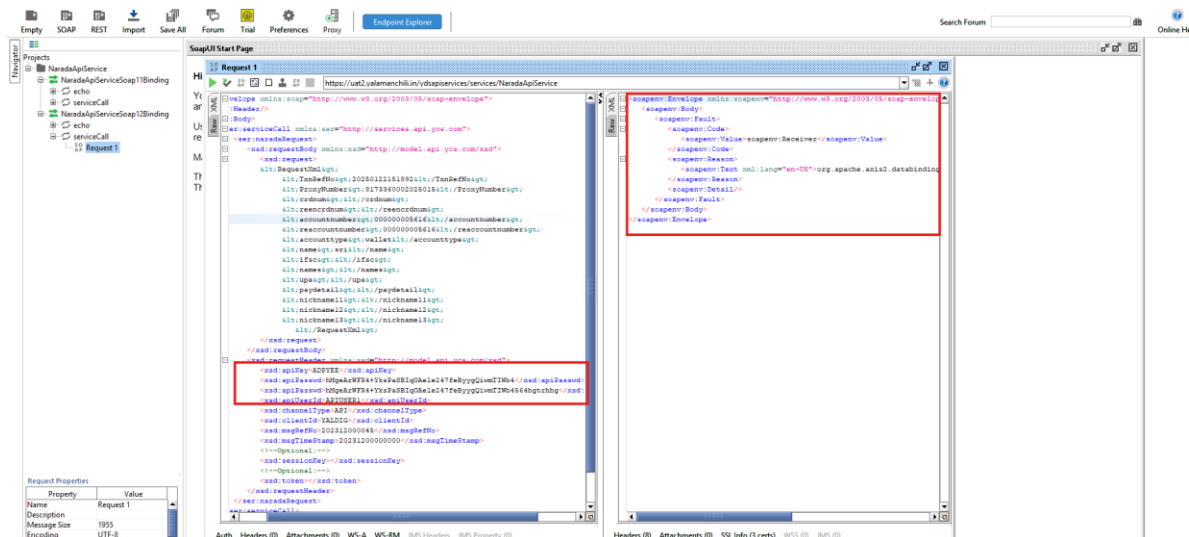


Image 2: It is observed that in response it shows error message.



Expected Result:

If the username has predefined limits, any incorrect input will trigger an exception, enhancing the application's security. However, if the limit can be bypassed, the case remains open; otherwise, it is considered closed.

Mitigation:

The parameters should be defined with specific boundaries, including minimum and maximum lengths. For example, the **username** parameter should have a minimum length of **5** and a maximum length of **35**, based on the requirement ensuring efficient validation during processing. This helps prevent the system from being overwhelmed by overly large or malicious inputs and ensures data consistency. Additionally, enforce input validation and implement rate limiting to further reduce potential attack vectors.

5.4 WSDL disclosure

Test Case Name: WSDL disclosure

Severity: High

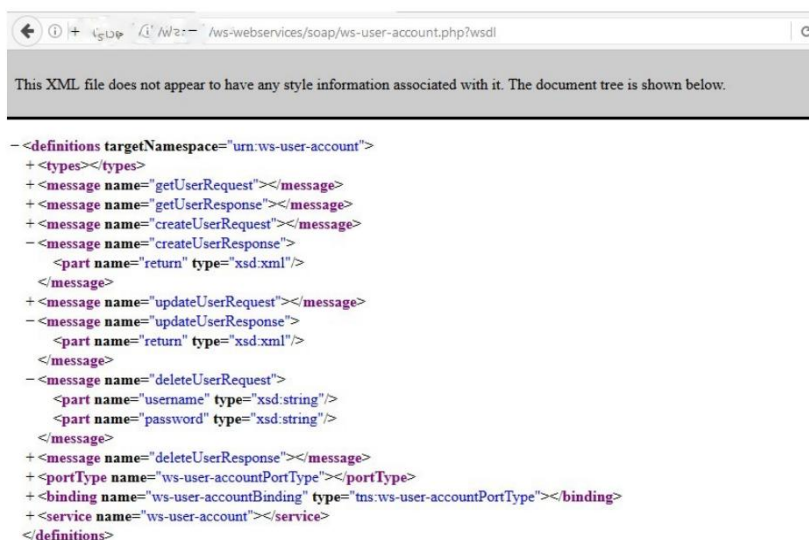
Description:

While WSDL disclosure is not an attack, it serves as a crucial step in facilitating one. Since WSDL files contain detailed information about web services, certain sensitive services—such as payment gateways or those handling confidential data—should be protected from unauthorized access. Attackers can easily locate exposed web services by conducting a simple search using `inurl:?wsdl` in search engines, making it essential to restrict public access to such files.

Test Procedure:

1. Copy the Inbound (SOAP) API URL.
2. Enter the URL in browser.
3. Check whether the URL discloses any sensitive data.

Image: Here we enter the URL in web browser. It is observed that there is no sensitive data



Expected Result:

If the URL contains sensitive data, the case remains open for further investigation. However, if no sensitive information is found, the case is considered closed.

Mitigation:

Implement strict access controls to ensure only authorized users can retrieve the WSDL file. Restrict or disable public access to WSDL endpoints unless explicitly needed. Strengthen security with authentication measures and encrypt WSDL transmissions using TLS/SSL to prevent unauthorized access. Remove sensitive details from the WSDL to minimize exposure of internal system information. Conduct regular audits and updates to ensure the WSDL contains only essential data.

5.5 Payload Encryption

Test Case Name: Payload Encryption

Severity: Critical

Description:

Payload encryption is the process of securing data by converting it into an unreadable format during transmission or storage to protect sensitive information. It ensures that only authorized parties can decrypt and access the data

Test Procedure:

1. Verify Payload Encryption is enabled or not

Expected Result:

Verify which encryption algorithm is used. We need to verify the algorithm is approved as per our standards.

Mitigation:

Using strong encryption algorithms and implementing key management best practices to enhance security.

6. Rest API Test Cases

6.1 Test for HTTP methods

Test Case Name: Test for HTTP methods

Severity: High

Description:

HTTP requests can include methods beyond GET and POST. Support for other verbs may disclose sensitive information (for example, TRACE) or allow for a dangerous invocation of application code (for example, DELETE). Let's see how Burp can help test for HTTP verb tampering.

Procedure:

- Add the Json Collection in Postman or the endpoint (URL) Using Post Man tool
- Now Set the Request along with the header values.
- Now alter the POST/GET request to PUT, UPDATE, INSERT, DELETE, PATCH, COPY, etc.
- Otherwise capture the API request in burp suite
- Make change in response and send the request
- Now observe the response, if it shows 200 O.K, then case is open

Image 1: It is Observe that we change the request from POST to Get and capture it.

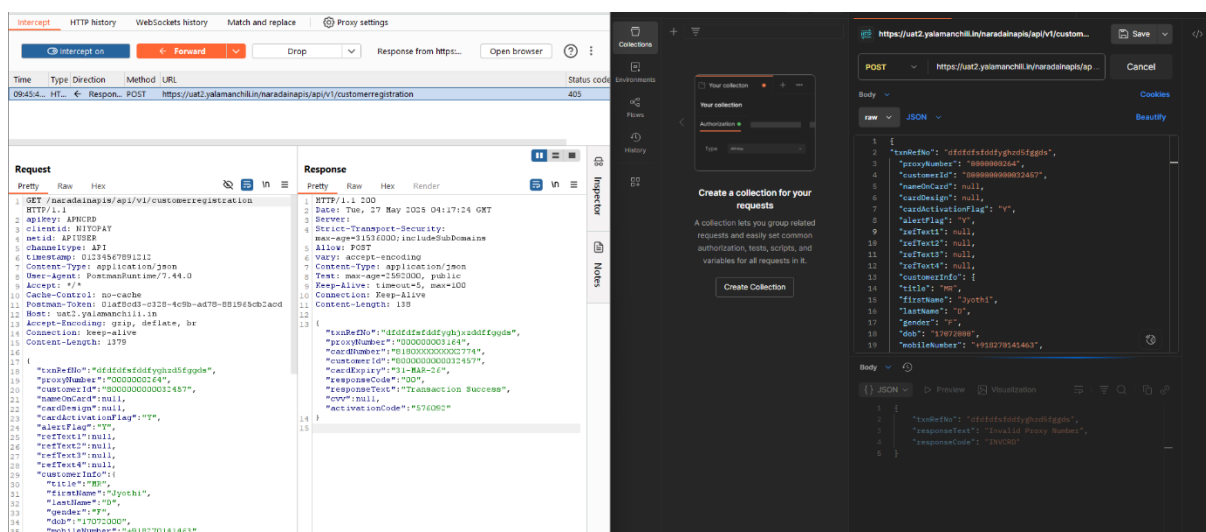
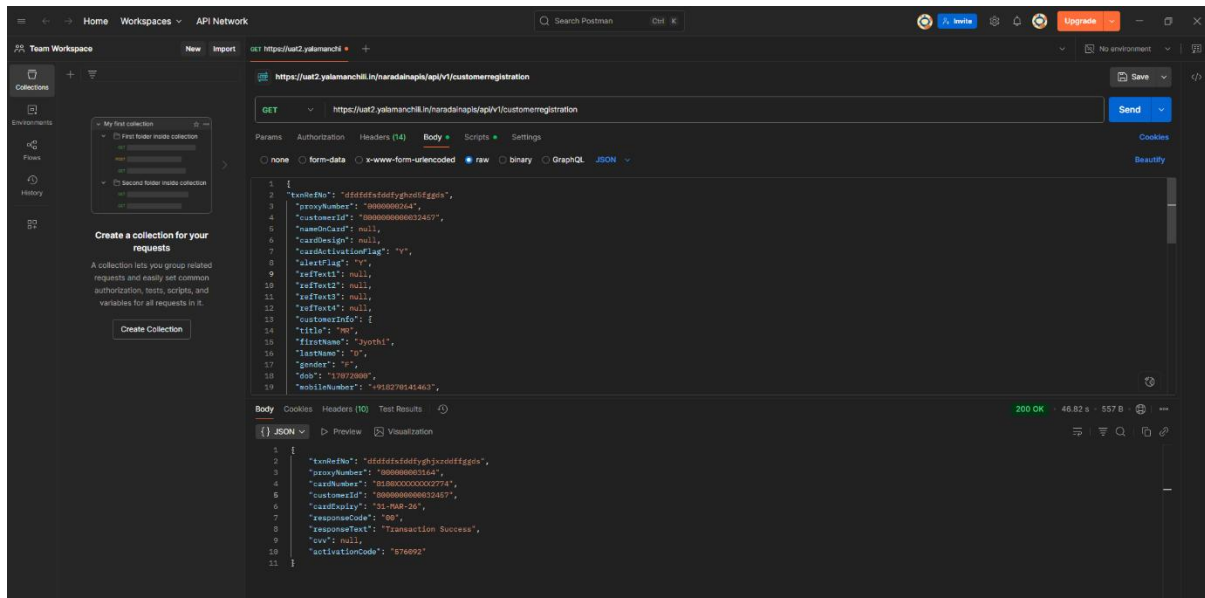


Image 2: Once the request is captured, modify the request in the response tab and then proceed with sending the request. If system returns 200 OK, it is indicating a successful operation



Expected Result:

If the response returns 200 OK, the case remains open. However, if a different response is received, the case is considered closed.

Mitigation:

Only GET, POST methods must be enabled in the server

6.2 Clear text Transmission of Sensitive Data

Test Case Name: Clear text Transmission of Sensitive Data

Severity: **Critical**

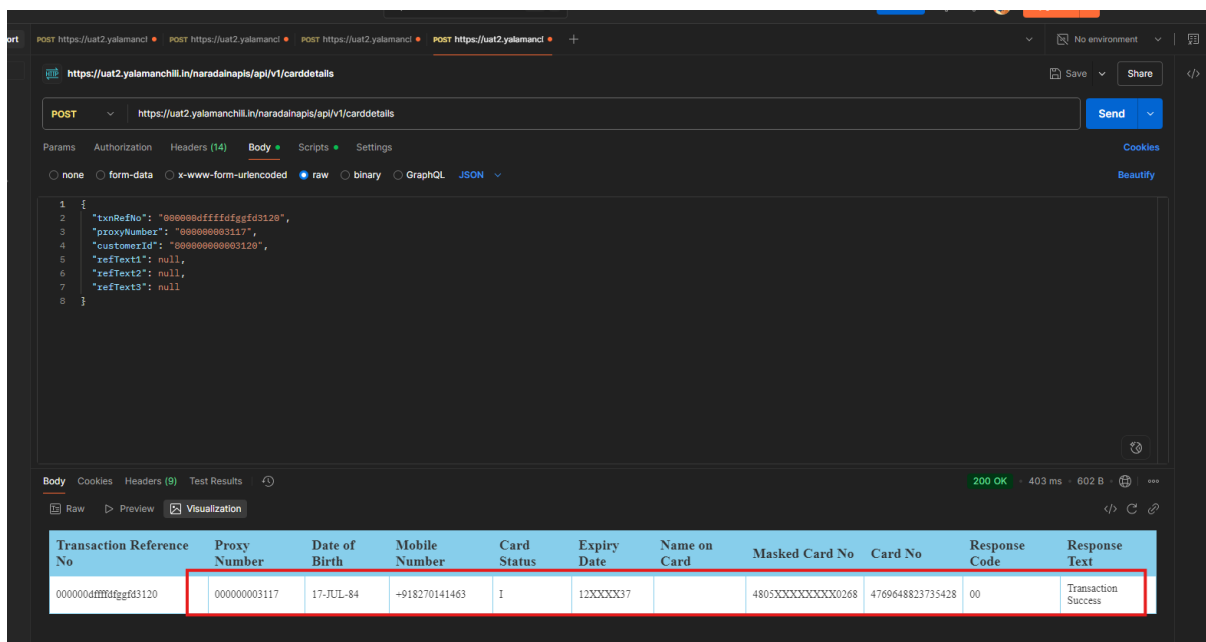
Description:

Sensitive data such as card number, Customer Information and other details are communicating in clear text. The information might help an attacker gain a greater understanding of the user details and potentially develop further attacks targeted person.

Test Procedure:

- Add the Json Collection in Postman or the endpoint (URL) Using Post Man tool
- Now Set the Request along with the header values.
- Now we verify the response.

Image: It is observed that the response is in clear text format.



Transaction Reference No	Proxy Number	Date of Birth	Mobile Number	Card Status	Expiry Date	Name on Card	Masked Card No	Card No	Response Code	Response Text
000000dffffd3120	00000003117	17-JUL-84	+918270141463	I	12XXXXX37		4805XXXXXXXXXX0268	4769648823735428	00	Transaction Success

Expected Result:

If the response is in clear text, the case remains open. However, if the data is obscured or encrypted, the case is considered closed.

Mitigation:

User Information should be protected by using hashing or encryption throughout the application

6.3 Performance / No Rate Limit Test

Test Case Name: Performance / No Rate Limit Test

Severity: **Critical**

Description:

A **No Rate Limit** vulnerability occurs when an application fails to limit the number of requests a user or attacker can make within a specific time period. This can allow attackers to overwhelm the system with excessive requests, leading to Denial of Service (DoS) or brute force attacks. Proper rate limiting is essential to protect applications from abuse, ensure performance, and prevent unauthorized access.

Procedure:

- Collect the Json file from development team
- Upload the Json file or csv which contain request response data
- Now Set the iteration and delay count
- Run the scan

Image: Upload the file and run the scan.

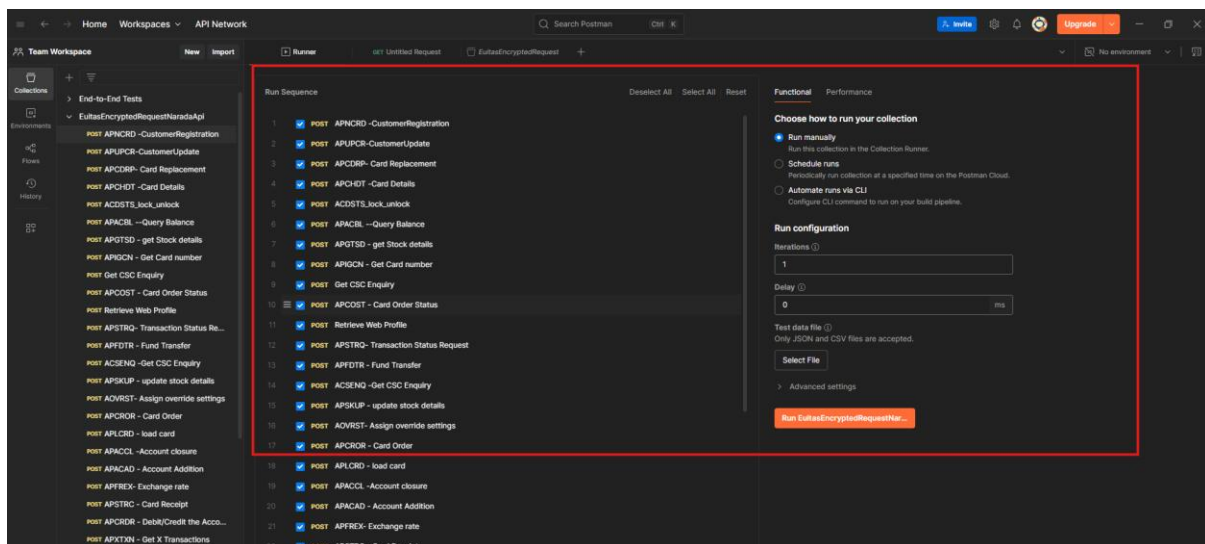
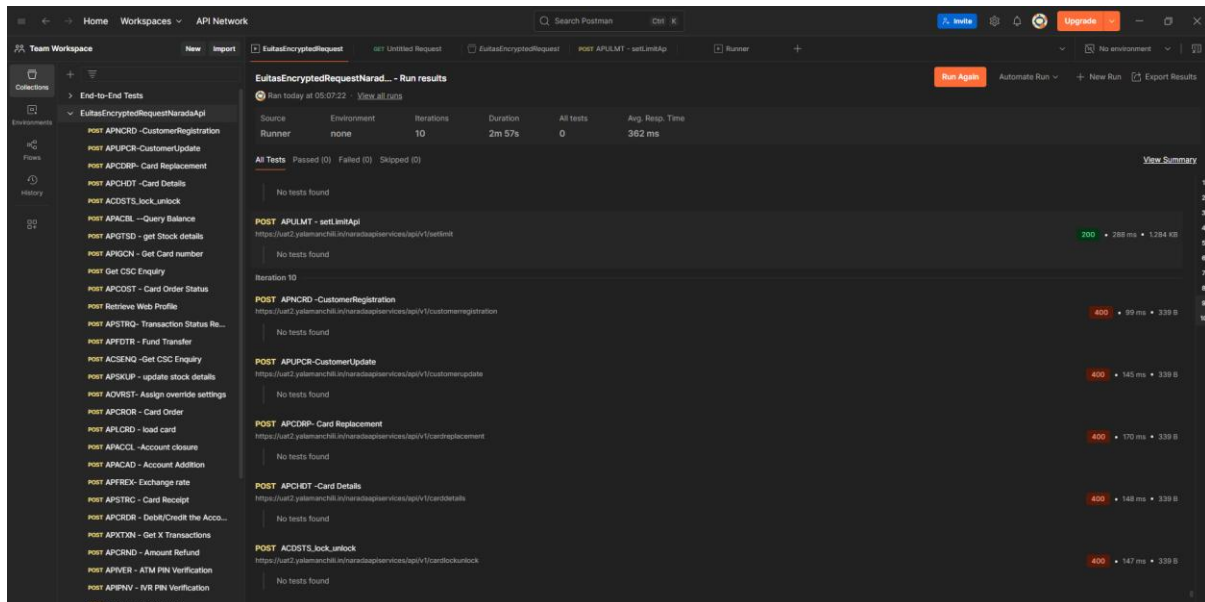


Image 2: Here it is observed that 403 & 200 error.



Expected Result:

Here we need to verify the response is there any leakage of sensitive data. it's essential to verify that each request and response is functioning correctly. This includes ensuring data integrity, checking for unintended exposure, and confirming that all security protocols are properly in place.

Mitigation:

To prevent the absence of rate limits in REST APIs, enforce request restrictions based on user, IP, or API key to control excessive access. Utilize an API gateway or middleware to apply these limits, ensuring responses include HTTP 429 Too Many Requests along with Retry-After headers. Implement stricter thresholds for sensitive endpoints such as login or registration to enhance security. Regularly monitor traffic patterns and log violations to detect anomalies and mitigate potential abuse.

7. List of Common Test case for Both Inbound (SOAP)API & Rest API.

List of Test Cases:

- Sonar/Yasca Report
- SCA & SBOM report
- Analysis for error codes
- Keystore path must not be logged in the log files
- Session keys and tokens must not be captured in the log files
- Cryptographic algorithms must not be logged in log files
- Encrypted Passwords must not be logged
- Analysis for Stack Traces
- Sensitive data must not be logged in the log files.
- Authentication Audit trail
- Common Audit Trail
- Sensitive Data must not be stored in Audit tables
- Password Configuration
- Password encryption in API
- API Authentication
- API Message Integrity
- Presence of Backup files/folders
- jQuery Version
- JDK Version
- Tomcat Version
- Usage of weak cryptographic algorithms
- Usage of Proprietary algorithms
- Keystore Formats
- Storage Path of Keystore
- Keystore Password Configuration
- Keys maintenance for client
- Configuration of Keys
- Configuration of Private Key password
- Storage of PKI Keys
- Storage of Symmetric Keys
- Keys Maintenance

Key Points For testing API:

- Based on the API functionality We will execute test cases based on industry standards like **PCI-DSS, ISO/IEC 27001, OWASP API Security Top 10, and SOC 2** to ensure API security and compliance.
- During testing, modifying the transaction reference number is essential to maintain accuracy and prevent conflicts.
- In API Testing every error or response is displayed through codes 200,403,00,500
- We Must collect API Specification document to check API functionalities, Error codes etc...